ESHRAGH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Stride Scheduling Algorithm: Principles, Implementation, Performance Analysis, and Comparison with Other CPU Scheduling Techniques

A Comprehensive Academic Research Paper on Deterministic Proportional-Share CPU Scheduling

STUDENT INFORMATION

Full Name: *Zeynab Rezaii*
Student ID: *1403176010*
Course Name: *Majid Shamshirgaran*

INSTRUCTOR & SUBMISSION

Instructor: *Majid Shamshirgaran*
Submission Date: *10/22/2026*

Prepared as a formal academic submission · All illustrative simulations are clearly labeled

Table of Contents

1. Abstract and Keywords
2. Introduction
3. Background and Related Concepts
4. Overview of Stride Scheduling
5. Mathematical Model and Algorithm Formulation
6. Step-by-Step Worked Example
7. Pseudocode and Algorithm Implementation
8. Flowchart of Stride Scheduling
9. Multiprocessing and Multicore Considerations
10. Features and Advantages
11. Limitations, Disadvantages, and Potential Defects
12. Starvation and Fairness
13. Deadlock and Its Relationship to Stride Scheduling
14. Performance Analysis and Complexity
15. Charts and Graphical Analysis
16. Comparison with Other Scheduling Algorithms
17. Applications and Practical Use Cases
18. Experimental Evaluation
19. Possible Improvements and Future Research
20. Conclusion
21. References

CHAPTER 1

Abstract and Keywords

Abstract

Stride Scheduling is a deterministic proportional-share CPU scheduling algorithm introduced by Carl A. Waldspurger and William E. Weihl in the mid-1990s as a complement to their earlier randomized lottery scheduling mechanism. The algorithm addresses a fundamental limitation of conventional priority-based schedulers: the inability to provide precise, controllable, and long-term proportional allocation of processor time among competing clients[reference:0]. Stride Scheduling achieves this by assigning each runnable process a *ticket* count that represents its relative resource entitlement, deriving from each ticket count a *stride* value that is inversely proportional to the ticket allocation, and maintaining a *pass* value that tracks the virtual progress of each process. At every scheduling decision, the runnable process with the minimum pass value is selected for execution, and its pass value is advanced by its stride[reference:1]. This deterministic mechanism guarantees that, over a sufficiently long interval, each process receives a CPU share approximately proportional to its ticket count. The paper provides a comprehensive analysis of Stride Scheduling, including its mathematical foundations, a fully worked numerical example, language-independent pseudocode, a detailed flowchart, multiprocessor considerations, performance complexity analysis, and an extensive comparison with Round Robin, FCFS, Priority Scheduling, Completely Fair Scheduling (CFS), Rate Monotonic, and Earliest Deadline First. The paper distinguishes proportional-share scheduling from deadline-based real-time scheduling, examines starvation and deadlock properties, and presents a reproducible simulated experimental evaluation. The principal conclusion is that Stride Scheduling offers a mathematically elegant and practically effective mechanism for weighted CPU sharing, but that its advantages depend critically on correct ticket allocation, appropriate initialization policies, and careful adaptation for multicore environments.

Keywords: Stride Scheduling, Proportional-Share Scheduling, Lottery Scheduling, CPU Scheduling, Operating Systems, Fairness, Pass Value, Ticket Allocation, Completely Fair Scheduler, Real-Time Systems, Starvation, Deadlock, Multiprocessor Scheduling.

Research Objectives

- To explain the operating principle of Stride Scheduling with mathematical precision and consistent notation.
- To demonstrate the algorithm through a reproducible hand-calculated simulation and to analyze its fairness properties.
- To provide implementation guidance through pseudocode and a complete flowchart.
- To examine the algorithm's strengths and limitations honestly, distinguishing inherent trade-offs from implementation defects.
- To situate Stride Scheduling within the broader landscape of CPU scheduling algorithms, including general-purpose, proportional-share, and real-time schedulers.
- To identify open research questions, particularly concerning multiprocessor fairness and dynamic ticket allocation.

Scope of the Paper

The paper focuses on Stride Scheduling as a uniprocessor proportional-share CPU scheduling algorithm and extends the discussion to multiprocessor considerations where appropriate. It does not claim that Stride Scheduling is a universal solution to all scheduling problems; rather, it analyzes the conditions under which the algorithm is appropriate and the conditions under which its guarantees weaken. The paper is intended for undergraduate computer science students, university instructors, and readers familiar with basic operating systems concepts.

Main Contributions and Findings

The paper makes several contributions. First, it consolidates the original research on Stride Scheduling into a single coherent pedagogical treatment with consistent mathematical notation. Second, it provides a fully worked numerical example that can be reproduced by hand or by simulation, thereby bridging the gap between theory and practice. Third, it presents a detailed complexity analysis of different implementation strategies, including linear scanning, priority queues, and balanced search trees. Fourth, it offers a structured comparison with seven alternative scheduling algorithms, carefully distinguishing proportional-share objectives from real-time objectives. Fifth, it analyzes starvation and deadlock properties with appropriate qualifications, avoiding the overstatement that Stride Scheduling makes starvation impossible in all circumstances. The main finding is that Stride Scheduling provides strong long-term proportional fairness under stable workload assumptions, but that its short-term behavior and multicore behavior require careful design.

CHAPTER 2

Introduction

Background of CPU Scheduling

CPU scheduling is one of the central responsibilities of an operating system. When multiple processes or threads are runnable, the scheduler must decide which one receives the processor, for how long, and in what order. The choice of scheduling policy affects system throughput, response time, fairness, predictability, and the degree to which the system can honor quality-of-service commitments. Traditional scheduling algorithms such as First-Come, First-Served (FCFS), Shortest Job First (SJF), Round Robin (RR), and priority scheduling were designed primarily for general-purpose time-sharing systems and for optimizing metrics such as average waiting time or average turnaround time. These algorithms, however, do not provide a direct mechanism for specifying that one client should receive, for example, twice as much CPU time as another client[reference:2].

Importance of Processor Allocation

Processor allocation is important because the CPU is a scarce and shared resource. In multi-user systems, server environments, virtualized infrastructures, and multi-tenant platforms, different clients may have different entitlements or different willingness to pay for computational capacity. A scheduler that ignores these differences may produce allocations that are perceived as unfair or that violate service-level agreements. Conversely, a scheduler that supports explicit proportional shares allows system administrators and application developers to express resource entitlements directly, rather than indirectly through priority values whose relationship to actual CPU share is often opaque.

Motivation for Proportional-Share Scheduling

Proportional-share scheduling emerged as a response to the limitations of conventional priority schedulers. The key idea is that each client should be able to specify a relative share of a resource, and the scheduler should allocate the resource in proportion to those shares. Waldspurger and Weihl introduced lottery scheduling as a randomized mechanism for achieving this goal, and Stride Scheduling as a deterministic alternative[reference:3]. The deterministic nature of Stride Scheduling is important because it eliminates the stochastic variance inherent in lottery scheduling, making short-term behavior more predictable while preserving long-term proportional fairness.

Research Problem and Objectives

The research problem addressed by this paper is twofold. First, there is a pedagogical problem: the original descriptions of Stride Scheduling are distributed across technical reports and theses, and a consolidated, mathematically careful, and implementation-oriented treatment is valuable for students and instructors. Second, there is an analytical problem: the properties of Stride Scheduling—particularly its fairness guarantees, its behavior under dynamic process arrival and departure, and its adaptation to multiprocessor systems—are often summarized in overly simple terms. This paper aims to provide a nuanced analysis that distinguishes what the algorithm guarantees from what it does not guarantee.

Research Questions

1. What is the precise mathematical relationship between ticket allocation, stride value, pass value, and CPU share in Stride Scheduling?
2. Under what assumptions does Stride Scheduling provide long-term proportional fairness?
3. What are the principal implementation choices, and how do they affect time complexity?
4. How does Stride Scheduling behave in multiprocessor and multicore environments?
5. What are the conditions under which starvation-like behavior can occur, and how can it be mitigated?
6. How does Stride Scheduling compare with Round Robin, FCFS, Priority Scheduling, CFS, Rate Monotonic, and EDF?

Organization of the Paper

The remainder of the paper is organized as follows. Chapter 3 reviews background concepts in operating systems and scheduling. Chapter 4 provides an overview of Stride Scheduling. Chapter 5 presents the mathematical model. Chapter 6 provides a step-by-step worked example. Chapter 7 presents pseudocode, and Chapter 8 provides a complete flowchart. Chapter 9 discusses multiprocessing considerations. Chapters 10 and 11 analyze features, advantages, limitations, and potential defects. Chapters 12 and 13 examine starvation, fairness, and deadlock. Chapter 14 analyzes performance and complexity. Chapter 15 presents charts and graphical analysis. Chapter 16 compares Stride Scheduling with other algorithms. Chapter 17 discusses applications, Chapter 18 presents an experimental evaluation, Chapter 19 discusses improvements and future research, and Chapter 20 concludes. Chapter 21 provides the references.

CHAPTER 3

Background and Related Concepts

Operating System Processes and Threads

A process is an executing program instance with its own address space, resources, and execution state. A thread is a unit of execution within a process that shares the process's address space but maintains its own program counter, stack, and register state. From a scheduling perspective, the operating system schedules threads or processes depending on the kernel's threading model. In this paper, the term *process* is used generically to refer to a schedulable entity, although the same principles apply to kernel-level threads.

CPU Scheduling and Scheduling Policies

CPU scheduling is the activity of selecting which runnable entity will execute on a processor. A scheduling policy defines the rules for this selection. Policies can be classified along several dimensions: preemptive versus non-preemptive, work-conserving versus non-work-conserving, and fairness-oriented versus priority-oriented. Stride Scheduling is a preemptive, work-conserving, proportional-share policy: it selects among runnable processes, and a running process can be preempted at the end of its scheduling quantum if another process has a lower pass value.

Preemptive and Non-Preemptive Scheduling

In non-preemptive scheduling, a process that begins execution continues until it voluntarily yields the CPU, blocks, or terminates. In preemptive scheduling, the operating system can interrupt a running process and switch to another process. Preemption is essential for proportional-share scheduling because it allows the scheduler to enforce share entitlements even when a process would prefer to continue running. Stride Scheduling is typically implemented as a preemptive algorithm with a fixed or variable scheduling quantum.

Time Slices and Context Switching

A time slice, or quantum, is the interval for which a process is allowed to run before the scheduler considers switching to another process. Context switching is the act of saving the state of the current process and restoring the state of the next process. Context switches introduce overhead, so the quantum length represents a trade-off between responsiveness and efficiency. In Stride Scheduling, the pass value is updated after each quantum, and the quantum length therefore affects the granularity of fairness.

Fairness and Resource Allocation

Fairness in scheduling can be defined in several ways. *Proportional fairness* requires that each client receive a share of the resource proportional to its entitlement. *Max-min fairness* seeks to maximize the minimum allocation. *Long-term fairness* considers the allocation over an extended interval, while *short-term fairness* considers instantaneous or small-interval allocations. Stride Scheduling is designed primarily to provide long-term proportional fairness, with short-term deviations that are bounded by the stride values and the scheduling quantum.

Proportional-Share Scheduling

Proportional-share scheduling is a family of algorithms that allocate resources in proportion to client-specified shares. The shares are often represented by weights, tickets, or credits. Lottery scheduling uses tickets and probabilistic selection; Stride Scheduling uses tickets and deterministic selection based on pass values. Other proportional-share mechanisms include weighted fair queuing and deficit round robin. The key distinction between proportional-share scheduling and conventional priority scheduling is that priority values are typically ordinal, whereas shares are cardinal and have a direct quantitative interpretation[reference:4].

Static Priorities versus Proportional Resource Shares

Static priority scheduling assigns each process a fixed priority, and the scheduler always selects the highest-priority runnable process. The relationship between priority and CPU share is indirect and depends on the workload. In contrast, proportional-share scheduling allows a client to specify a share directly, and the scheduler attempts to realize that share over time. This distinction is important: a high-priority process in a priority scheduler may receive 100% of the CPU if it never blocks, whereas a high-share process in a proportional-share scheduler receives only its specified proportion, leaving capacity for other clients.

Scheduling Algorithms and Real-Time Systems

Real-time systems impose timing constraints on computations. Deadline-based real-time scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), are designed to ensure that tasks meet their deadlines. These algorithms have different assumptions and objectives from proportional-share scheduling. RM assigns fixed priorities based on task periods; EDF assigns dynamic priorities based on absolute deadlines. Neither algorithm is designed to provide proportional CPU shares, and Stride Scheduling is not designed to provide deadline guarantees. This distinction is maintained throughout the paper to avoid conflating proportional-share scheduling with real-time scheduling[reference:5].

Table 1. Summary of Stride Scheduling Terminology

Term	Definition	Role in the Algorithm
Ticket	A relative entitlement value assigned to a process.	Determines the process's CPU share; higher ticket count implies a larger share.
Stride	The interval by which a process's pass value advances after each quantum.	Computed as `K / tickets`; inversely proportional to tickets.
Pass value	A virtual-time counter that tracks the process's accumulated service.	The scheduler selects the runnable process with the minimum pass value.
Stride constant (K)	A configurable large integer used to scale stride values.	Controls numerical precision and the magnitude of pass values.
Quantum	The CPU time granted to a selected process before the next scheduling decision.	Determines how often pass values are updated and how granular the fairness is.
Global pass	The minimum pass value among runnable processes.	Used as a reference for initialization and for detecting imbalance.

CHAPTER 4

Overview of Stride Scheduling

Definition and Purpose

Stride Scheduling is a deterministic proportional-share resource allocation algorithm. Its purpose is to allocate a resource—most commonly CPU time—among competing clients in proportion to their ticket allocations. Unlike lottery scheduling, which selects a client probabilistically, Stride Scheduling makes a deterministic decision at each scheduling point by selecting the runnable client with the smallest pass value[reference:6]. This determinism provides more predictable short-term behavior and simplifies reasoning about the algorithm's fairness properties.

Historical Background and Original Research Context

Stride Scheduling was developed by Carl A. Waldspurger and William E. Weihl at the Massachusetts Institute of Technology in the context of their research on flexible proportional-share resource management. Lottery scheduling was presented at the First Symposium on Operating Systems Design and Implementation in 1994[reference:7]. Stride Scheduling was described in Waldspurger's 1995 doctoral thesis and in a technical memorandum[reference:8]. The algorithm cross-applies and generalizes elements of rate-based flow control algorithms designed for networks to the problem of scheduling processor time and other resources[reference:9]. The original work also introduced hierarchical stride scheduling, which supports composition of resource allocations[reference:10].

Design Objectives

- **Proportional sharing:** Allocate CPU time in proportion to ticket counts.
- **Determinism:** Avoid the stochastic variance of lottery scheduling.
- **Low overhead:** Support efficient scheduling decisions for large numbers of clients.
- **Flexibility:** Allow tickets to be issued, transferred, and grouped into currencies.
- **Generality:** Apply to resources beyond CPU time, including memory, locks, and disk bandwidth.

Core Principles

The core principle of Stride Scheduling is that each process should advance through a virtual timeline at a rate determined by its ticket allocation. A process with many tickets advances slowly through virtual time and therefore receives more frequent service. A process with few tickets advances quickly and receives less frequent service. The scheduler always selects the process that is furthest behind in virtual time, as indicated by the minimum pass value.

Ticket Allocation

Tickets represent relative resource entitlements. If process A has 100 tickets and process B has 50 tickets, then A is entitled to twice as much CPU time as B, assuming both are continuously runnable. Tickets may be assigned statically at process creation or adjusted dynamically. The original work also describes ticket transfers and ticket currencies, which allow flexible composition of resource-management policies[reference:11].

Stride Calculation

The stride of a process is inversely proportional to its ticket count. The formula is:

EQUATION 1 — STRIDE CALCULATION

$$\text{stride}_i = K / \text{tickets}_i$$

where stride_i is the stride assigned to process i, K is a configurable large integer constant, and tickets_i is the number of tickets held by process i. A larger ticket count produces a smaller stride, which means that the process's pass value increases more slowly, allowing it to be selected more frequently.

Pass Values

Each process maintains a pass value that represents its virtual progress. The pass value is initialized before the process first runs and is updated after each quantum according to:

EQUATION 2 — PASS-VALUE UPDATE

$$\text{pass}_i = \text{pass}_i + \text{stride}_i$$

The scheduler selects the runnable process with the minimum pass value. This selection rule ensures that the process that has received the least service relative to its entitlement is chosen next.

Virtual Time and Scheduling Order

The pass values collectively define a virtual time axis. The process with the minimum pass value is the one whose virtual time is furthest behind the others. By advancing this process's pass value by its stride after it runs, the algorithm moves it forward in virtual time, allowing other processes to catch up. Over time, the pass values of continuously runnable processes remain approximately balanced, with differences bounded by the maximum stride value.

The Role of the Global Minimum Pass Value

The global minimum pass value, often denoted `global_pass`, is the minimum pass value among all runnable processes. It serves several purposes. First, it defines the current position of the virtual time frontier. Second, it can be used to initialize newly created or newly awakened processes so that they do not receive an unfair advantage or disadvantage. Third, it provides a reference for detecting and correcting pass-value drift in long-running systems.

Process Initialization and State Updates

When a process is created, its stride is computed from its ticket allocation, and its pass value must be initialized. A common policy is to set the new process's pass value to the current global minimum pass value, or to the pass value of the process that created it. This policy prevents a new process from monopolizing the CPU by starting with an artificially low pass value. When a process blocks and later wakes up, its pass value may need to be ad-

justed to reflect the service it missed while it was not runnable. The original work discusses these issues and proposes mechanisms for handling dynamic changes in the client set[reference:12][reference:13].

Ticket Count, Stride Value, and Pass Value

The distinction between ticket count, stride value, and pass value is fundamental. The ticket count is the input that expresses entitlement. The stride value is derived from the ticket count and determines the rate at which the pass value advances. The pass value is the state variable that the scheduler uses to make decisions. Confusing these concepts leads to misunderstandings about the algorithm. For example, a process with a high ticket count has a small stride but not necessarily a small pass value at any given moment; its pass value depends on its execution history.

CHAPTER 5

Mathematical Model and Algorithm Formulation

Formal Definitions

Let $P = \{p_1, p_2, \ldots, p_n\}$ be the set of runnable processes. Each process p_i has a ticket count $t_i > 0$, a stride $s_i = K / t_i$, and a pass value v_i. The scheduling quantum is denoted q. The algorithm proceeds in discrete scheduling steps. At each step, the process with the minimum pass value is selected:

EQUATION 3 — PROCESS SELECTION

$$\text{selected} = \arg\min_{p_i \in P} v_i$$

After the selected process runs for one quantum, its pass value is updated:

EQUATION 4 — PASS-VALUE ADVANCEMENT

$$v_{\text{selected}} \leftarrow v_{\text{selected}} + s_{\text{selected}}$$

The process remains in the runnable set if it is still runnable, and the next scheduling decision is made.

The Initial-Pass Problem

The initial pass value assigned to a process affects the algorithm's transient behavior. If all processes start with the same pass value, the algorithm begins in a balanced state. If a new process starts with a pass value of zero while existing processes have large pass values, the new process will be selected repeatedly until its pass value catches up, potentially causing a burst of unfairness. Conversely, if a new process starts with a pass value equal to the current global minimum, it joins the competition fairly. The choice of initialization policy is therefore a design decision with significant practical consequences[reference:14].

Effects of Different Values of K

The stride constant K controls the granularity of the stride values. A large K produces large stride values, which may lead to coarse-grained pass-value updates and increased quantization error. A small K produces small stride values, which can lead to rapid pass-value growth and potential overflow. In practice, K is chosen to be large enough to provide sufficient precision but small enough to avoid numerical problems. The original implementation uses fixed-point integer arithmetic and discusses the accumulation of quantization errors[reference:15].

Numerical Precision and Tie-Breaking

Because stride values may not be integers for all ticket allocations, fixed-point or floating-point arithmetic is required. Rounding introduces small errors that can accumulate over time. Tie-breaking is another practical issue: when two or more processes have the same minimum pass value, the scheduler must choose one. Common poli-

cies include selecting the process with the smaller stride (higher ticket count), selecting the process that has been waiting longer, or using a deterministic rule based on process identifier. The tie-breaking policy can affect short-term fairness but should not affect long-term proportionality if the tie is broken consistently.

Assumptions Behind the Formulas

The formulas assume that tickets are positive integers, that the quantum is fixed, and that the set of runnable processes is stable during the interval under consideration. When processes arrive, depart, block, or wake up, additional mechanisms are required to maintain fairness. The formulas also assume that the cost of scheduling decisions is negligible relative to the quantum; if the quantum is very small, scheduling overhead can become significant.

Worked Numerical Example

Consider three processes with ticket counts $t_1 = 100$, $t_2 = 50$, and $t_3 = 25$. Let $K = 10000$. The strides are:

- $s_1 = 10000 / 100 = 100$
- $s_2 = 10000 / 50 = 200$
- $s_3 = 10000 / 25 = 400$

Suppose all processes start with pass value 0. The first scheduling decision selects P1 because all pass values are equal and P1 has the smallest stride (or by an arbitrary tie-break). After P1 runs, its pass value becomes 100. The next decision selects P1 again (pass 100 vs. 0 and 0) — wait, this is incorrect because P2 and P3 still have pass 0. The minimum pass value is 0, shared by P2 and P3. The scheduler selects one of them, say P2. After P2 runs, its pass becomes 200. The minimum is now P3 with pass 0, so P3 runs and its pass becomes 400. The next minimum is P1 with pass 100, so P1 runs and its pass becomes 200. The sequence continues. This example illustrates the basic mechanics and will be expanded in Chapter 6.

A process with more tickets generally receives a larger share because its stride is smaller, so its pass value increases more slowly, and it is selected more frequently. Over a long interval, the fraction of scheduling decisions awarded to process *i* approaches $t_i \,/\, \Sigma\, t_j$, provided that all processes remain runnable.

CHAPTER 6

Step-by-Step Worked Example

This chapter presents a complete, hand-calculated illustrative simulation. The data are not measurements from real hardware; they are derived directly from the mathematical model and are intended to demonstrate the algorithm's behavior. The example uses three processes with ticket counts 100, 50, and 25, and a stride constant `K = 10000`. The quantum is one abstract time unit. All processes are assumed to be continuously runnable throughout the simulation. Ties are broken by selecting the process with the smaller stride (higher ticket count), and if strides are equal, by process identifier.

Initial Process Table

Table 2. Process Ticket and Stride Calculations

Process	Tickets	Stride (K = 10000)	Initial Pass	Theoretical Share
P1	100	100	0	100/175 ≈ 57.14%
P2	50	200	0	50/175 ≈ 28.57%
P3	25	400	0	25/175 ≈ 14.29%

The total ticket count is 175. The theoretical CPU shares are proportional to ticket counts and sum to 100%.

Step-by-Step Scheduling Table

The following table traces the first 12 scheduling decisions. At each step, the process with the minimum pass value is selected, runs for one quantum, and has its pass value increased by its stride.

Table 3. Step-by-Step Scheduling Decisions (Illustrative Simulation)

Step	Pass Values Before (P1, P2, P3)	Selected	Pass Values After (P1, P2, P3)	Reason
1	(0, 0, 0)	P1	(100, 0, 0)	Tie; P1 has smallest stride
2	(100, 0, 0)	P2	(100, 200, 0)	Tie between P2 and P3; P2 has smaller stride
3	(100, 200, 0)	P3	(100, 200, 400)	P3 has minimum pass
4	(100, 200, 400)	P1	(200, 200, 400)	P1 has minimum pass
5	(200, 200, 400)	P1	(300, 200, 400)	Tie between P1 and P2; P1 has smaller stride
6	(300, 200, 400)	P2	(300, 400, 400)	P2 has minimum pass
7	(300, 400, 400)	P1	(400, 400, 400)	P1 has minimum pass
8	(400, 400, 400)	P1	(500, 400, 400)	Tie; P1 has smallest stride
9	(500, 400, 400)	P2	(500, 600, 400)	Tie between P2 and P3; P2 smaller stride
10	(500, 600, 400)	P3	(500, 600, 800)	P3 has minimum pass
11	(500, 600, 800)	P1	(600, 600, 800)	P1 has minimum pass
12	(600, 600, 800)	P1	(700, 600, 800)	Tie between P1 and P2; P1 smaller stride

CPU Execution Timeline

The execution timeline for the first 12 quanta is:

```
Quantum: 1 2 3 4 5 6 7 8 9 10 11 12
Process: P1 P2 P3 P1 P1 P2 P1 P1 P2 P3 P1 P1
```

Observed Execution Frequency

In the first 12 decisions, P1 runs 7 times, P2 runs 3 times, and P3 runs 2 times. The observed shares are 58.33%, 25.00%, and 16.67%, respectively. The theoretical shares are 57.14%, 28.57%, and 14.29%. The small deviation is expected in a short interval and illustrates the difference between theoretical long-term proportionality and short-term observed behavior.

Theoretical and Observed CPU Shares

Table 4. CPU-Share Distribution (Illustrative, 12 Quanta)

Process	Tickets	Theoretical Share	Observed Runs	Observed Share	Absolute Deviation
P1	100	57.14%	7	58.33%	1.19%
P2	50	28.57%	3	25.00%	3.57%
P3	25	14.29%	2	16.67%	2.38%

As the number of scheduling decisions increases, the observed shares converge toward the theoretical shares. The convergence rate depends on the stride values and the ticket distribution. This behavior is consistent with the algorithm's design objective of long-term proportional fairness.

> **Note on Reproducibility:** The simulation above uses deterministic rules and can be reproduced exactly. The tie-breaking policy and initialization policy are stated explicitly so that others can replicate the results.

CHAPTER 7

Pseudocode and Algorithm Implementation

This chapter presents language-independent pseudocode for the core Stride Scheduling algorithm. The pseudocode assumes a single-processor system with a set of runnable processes. Extensions for blocked and newly awakened processes are discussed after the core algorithm.

Process Creation and Ticket Assignment

```
function createProcess(pid, tickets, K):
    process.pid = pid
    process.tickets = tickets
    if tickets <= 0:
        // Handle zero or negative tickets according to policy
        raise InvalidTicketAllocation
    process.stride = K / tickets
    process.pass = globalPass()   // Initialize to current global minimum pass
    addToRunnableSet(process)
    return process
```

Selecting the Next Runnable Process

```
function selectNextProcess(runnableSet):
    if runnableSet is empty:
        return IDLE
    minPass = +infinity
    selected = null
    for each process in runnableSet:
        if process.pass < minPass:
            minPass = process.pass
            selected = process
        else if process.pass == minPass:
            // Tie-breaking: prefer smaller stride (more tickets)
            if process.stride < selected.stride:
                selected = process
    return selected
```

Updating the Selected Process

```
function updateSelectedProcess(process):
    process.pass = process.pass + process.stride
    // Update global pass reference if necessary
    updateGlobalPass()
```

Handling Blocked and Newly Awakened Processes

```
function blockProcess(process):
    removeFromRunnableSet(process)
    process.state = BLOCKED

function wakeProcess(process):
    // Adjust pass value to prevent starvation or unfair advantage
    process.pass = max(process.pass, globalPass())
    process.state = RUNNABLE
    addToRunnableSet(process)
```

Handling Process Termination

```
function terminateProcess(process):
    removeFromRunnableSet(process)
    process.state = TERMINATED
    // Reclaim resources; optionally redistribute tickets
```

Adding and Removing Processes from the Runnable Set

```
function addToRunnableSet(process):
    runnableSet.insert(process)

function removeFromRunnableSet(process):
    runnableSet.remove(process)
```

Time Complexity of Major Operations

Table 5. Implementation Complexity (Single-Processor)

Operation	Linear Scan	Min-Heap	Balanced BST
Select minimum pass	O(n)	O(1) peek / O(log n) extract	O(log n)
Insert process	O(1)	O(log n)	O(log n)
Remove process	O(n)	O(log n)	O(log n)
Update pass value	O(1) + O(n) if re-scan needed	O(log n) decrease/increase key	O(log n) delete + insert

The choice of data structure has a significant impact on the scheduler's scalability. A linear scan is simple to implement but becomes inefficient as the number of runnable processes grows. A min-heap or balanced search tree provides logarithmic operations and is preferable for large systems. The original work discusses these alternatives and their trade-offs[reference:16].

CHAPTER 8

Flowchart of Stride Scheduling

The following flowchart illustrates the complete Stride Scheduling cycle. It begins with system initialization, proceeds through process creation and ticket assignment, and then enters the main scheduling loop. The loop selects the process with the minimum pass value, executes it for one quantum, updates its pass value, and checks whether the process remains runnable, has blocked, or has terminated. The flowchart is implemented as inline SVG so that it remains visible when the document is printed to PDF.



Figure 1. Complete flowchart of the Stride Scheduling algorithm, including process creation, ticket assignment, stride calculation, pass-value initialization, scheduling loop, and state handling.

CHAPTER 9

Multiprocessing and Multicore Considerations

Multiprogramming, Multiprocessing, and Multithreading

Multiprogramming refers to the concurrent residency of multiple programs in memory, allowing the CPU to switch among them. *Multiprocessing* refers to the use of multiple physical or logical processors. *Multithreading* refers to multiple threads of execution within a process. Stride Scheduling, as originally formulated, is a uniprocessor algorithm. Extending it to multiprocessor systems requires additional design decisions.

Single CPU versus Multiple CPUs

On a single CPU, the scheduler maintains a single runnable set and a single global minimum pass value. On multiple CPUs, the scheduler must decide whether to maintain a global runnable set shared by all CPUs or per-CPU runnable sets. A global runnable set can provide global proportional fairness but introduces synchronization overhead and contention. Per-CPU runnable sets reduce contention but may produce fairness deviations when processes migrate between CPUs.

Global versus Per-CPU Run Queues

A global run queue with a single lock simplifies reasoning about fairness but becomes a scalability bottleneck. A per-CPU run queue with load balancing is more scalable but requires careful handling of pass values during migration. If a process moves from one CPU to another, its pass value must be adjusted to the new CPU's virtual time reference, or the scheduler must maintain a global pass-value reference that is periodically synchronized.

Synchronization and Concurrent Updates

In a multiprocessor system, multiple CPUs may attempt to select or update processes concurrently. The scheduling state must be protected by locks or lock-free data structures. Coarse-grained locking of a global run queue is simple but limits parallelism. Fine-grained locking or per-CPU structures improve scalability but increase complexity and the risk of subtle race conditions.

CPU Affinity and Load Balancing

CPU affinity restricts a process to a subset of CPUs. Affinity can improve cache locality but may conflict with proportional fairness if some CPUs are more heavily loaded than others. Load balancing attempts to equalize the number of runnable processes across CPUs. In a Stride Scheduling system, load balancing must also consider pass values: moving a process with a very low pass value to a CPU with higher pass values can give it an unfair burst of service.

Fairness Across Multiple Cores

Maintaining global proportional fairness across multiple cores is challenging. If each CPU independently runs Stride Scheduling on its local run queue, the aggregate allocation may approximate proportional fairness only if the workload is evenly distributed and migration is controlled. The original work notes that the basic algorithm

does not automatically provide correct global fairness on multiple processors; additional mechanisms are required[reference:17].

Scalability Challenges

As the number of cores increases, the cost of maintaining a global pass-value reference grows. Hierarchical stride scheduling, introduced in the original work, provides a way to compose allocations hierarchically and may help manage scalability[reference:18]. Research on proportional-share scheduling for multiprocessors has explored techniques such as global virtual time and performance-aware virtual time[reference:19].

> **Important Qualification:** A basic single-processor Stride Scheduling implementation does not automatically provide correct global proportional fairness on multiple processors. Additional design decisions concerning run-queue organization, synchronization, migration, and pass-value management are required.

CHAPTER 10

Features and Advantages

Proportional CPU-Share Allocation

The principal advantage of Stride Scheduling is its ability to allocate CPU time in proportion to ticket counts. This provides a direct and intuitive mechanism for expressing resource entitlements. The advantage is valid under the assumption that all processes are continuously runnable and that the ticket allocation reflects the desired shares.

Predictable Long-Term Resource Distribution

Because the algorithm is deterministic, its long-term behavior is predictable. Over a sufficiently long interval, the fraction of CPU time received by a process converges to its ticket proportion. This property is valuable in environments where consistent resource distribution is required.

Relative Fairness

Stride Scheduling provides relative fairness: it ensures that processes receive service in proportion to their tickets, rather than guaranteeing an absolute minimum share. This is appropriate for systems in which entitlements are defined relative to one another.

Configurable Resource Weights

Tickets serve as configurable weights. System administrators or application developers can adjust ticket allocations to reflect changing priorities or service-level agreements. The ticket abstraction is flexible and can be extended with transfers and currencies[reference:20].

Deterministic Scheduling Decisions

Under fixed assumptions, Stride Scheduling produces deterministic scheduling decisions. This determinism simplifies testing, debugging, and formal analysis. It also eliminates the stochastic variance of lottery scheduling, which can produce short-term fluctuations in allocation.

Relatively Simple Mathematical Foundations

The core formulas—stride = K / tickets and pass += stride—are simple and easy to implement. The algorithm's complexity lies primarily in the data structures used to maintain the runnable set and in the handling of dynamic process behavior, not in the core arithmetic.

Suitability for Simulations and Research

Stride Scheduling is well suited to simulation studies and resource-allocation research because its behavior is reproducible and its fairness properties are mathematically tractable. It also serves as a conceptual foundation for understanding later algorithms such as Linux's Completely Fair Scheduler, which uses a virtual runtime mechanism loosely based on the stride concept[reference:21].

Weighted Sharing in Multi-Tenant Environments

With appropriate adaptation, Stride Scheduling can be used in virtualized or multi-tenant environments to distribute CPU capacity according to tenant entitlements. However, the original algorithm design should be distinguished from modern adaptations that address the complexities of virtualization and multicore systems.

CHAPTER 11

Limitations, Disadvantages, and Potential Defects

Dependence on Correct Ticket Allocation

The fairness properties of Stride Scheduling depend on the ticket allocation being meaningful. If tickets are assigned incorrectly or if clients can manipulate their ticket counts, the resulting allocation may not reflect the intended policy. *Why it occurs:* Tickets are an input to the algorithm, not a property that the algorithm verifies. *Consequences:* Unfair or unintended CPU distribution. *Mitigation:* Access control on ticket assignment and careful policy design.

Numerical Precision Problems

Fixed-point or floating-point arithmetic introduces rounding errors that can accumulate over long periods. *Why it occurs:* Stride values may not be exactly representable. *Consequences:* Gradual drift in pass values and small fairness deviations. *Mitigation:* Use sufficiently large integer constants, periodic normalization, or exact rational arithmetic where feasible[reference:22].

Sensitivity to Initialization Policies

The choice of initial pass values affects transient behavior. A new process that starts with an artificially low pass value may monopolize the CPU until it catches up. *Mitigation:* Initialize new processes to the current global minimum pass value, or use a policy that accounts for the process's expected service.

Pass-Value Imbalance on Join or Wake-Up

When a process joins the runnable set after a long sleep, its pass value may be far below the current global pass. If it is allowed to run until it catches up, it may starve other processes temporarily. *Mitigation:* Clamp the pass value on wake-up to the global minimum or to a value that reflects the process's recent service[reference:23].

Scheduling Overhead

Selecting the minimum pass value requires a data structure operation. A linear scan is O(n) per decision, which is inefficient for large n. *Mitigation:* Use a min-heap or balanced search tree for O(log n) operations.

Tie-Breaking Behavior

When multiple processes have the same minimum pass value, the tie-breaking rule determines which one runs. An arbitrary or unfair tie-breaking rule can cause short-term unfairness. *Mitigation:* Use a consistent and documented tie-breaking policy.

Starvation-Related Edge Cases

Under incorrect implementations, starvation-like behavior can occur. For example, if a process's ticket count is zero, its stride is infinite (division by zero), and it may never be selected. *Mitigation:* Enforce positive ticket

counts and handle zero-ticket cases explicitly[reference:24].

Challenges in Multicore Systems

As discussed in Chapter 9, extending Stride Scheduling to multicore systems requires careful design. *Mitigation:* Use global or hierarchical pass-value management, appropriate synchronization, and load-balancing policies that account for pass values.

Lack of Inherent Deadline Guarantees

Stride Scheduling is not a real-time scheduling algorithm. It does not provide guarantees that a process will complete by a deadline. *Mitigation:* Use a real-time scheduler for deadline-sensitive tasks, or combine proportional-share and real-time mechanisms in a hierarchical framework.

Limitations in Interactive and Latency-Sensitive Workloads

Stride Scheduling focuses on proportional CPU share, not on minimizing response time. Interactive workloads may experience latency if their ticket allocation is low. *Mitigation:* Assign sufficient tickets to latency-sensitive processes or use a hybrid scheduler that considers both share and latency.

Theoretical versus Short-Term Behavior

The theoretical fairness guarantee is asymptotic. In short intervals, observed shares may deviate from theoretical shares. *Mitigation:* Choose appropriate quantum sizes and ticket allocations, and evaluate fairness over sufficiently long intervals.

Table 6. Advantages and Limitations of Stride Scheduling

Advantage	Condition for Validity	Corresponding Limitation
Proportional CPU share	Tickets reflect intended entitlements; processes are runnable	Incorrect ticket allocation leads to unintended shares
Deterministic decisions	Fixed workload and fixed tie-breaking policy	Determinism can produce predictable patterns that may be exploited
Long-term fairness	Sufficiently long interval; stable runnable set	Short-term deviations and initialization effects
Simple arithmetic	K chosen appropriately	Numerical precision and overflow issues
Flexible weights	Tickets can be adjusted	Dynamic ticket changes require careful state management

CHAPTER 12

Starvation and Fairness

Definition of Starvation

Starvation occurs when a runnable process is indefinitely prevented from making progress because other processes are repeatedly selected. In scheduling, starvation can occur if a low-priority or low-share process never receives CPU time.

Long-Term Service Guarantees

Under the assumptions that all ticket counts are positive, the runnable set is finite, and the scheduling quantum is finite, Stride Scheduling provides a long-term service guarantee: every continuously runnable process is selected infinitely often, and its long-term share approaches its ticket proportion. This property follows from the fact that the minimum pass value increases over time and no process can be permanently bypassed[reference:25].

Ticket Allocation and Service Received

A process with a very small ticket count has a large stride and therefore advances its pass value quickly. It will be selected less frequently than processes with larger ticket counts, but it will still be selected eventually if it remains runnable. Extremely small ticket counts can make the waiting time between selections very long, which may be perceived as starvation even though it is technically temporary.

The Role of Pass-Value Initialization

Incorrect initialization can create starvation-like behavior. If a new process starts with a pass value of zero while existing processes have large pass values, it may run for many consecutive quanta, temporarily starving the existing processes. Conversely, if a newly awakened process retains an old, very low pass value, it may also receive a burst of service.

Incorrect Process-State Handling

If a blocked process is not removed from the runnable set, or if a terminated process is not removed, the scheduler may make incorrect decisions. These are implementation bugs rather than inherent limitations of the algorithm.

Starvation, Unfairness, and Temporary Waiting

Starvation is distinct from unfairness and temporary waiting. Unfairness refers to a deviation from the intended proportional allocation. Temporary waiting is a normal consequence of sharing a resource. Starvation implies indefinite postponement. In a correct Stride Scheduling implementation with positive tickets and proper state management, indefinite starvation of a continuously runnable process should not occur, but long waiting times are possible for very low-ticket processes.

Table 7. Starvation and Mitigation Strategies

Condition	Cause	Consequence	Mitigation
Zero tickets	Invalid ticket allocation	Division by zero; process never selected	Enforce positive tickets; handle zero-ticket policy explicitly
Very low tickets	Large stride	Long waiting time; perceived starvation	Set a minimum ticket allocation or use a latency-aware policy
New process with zero pass	Incorrect initialization	Burst of service; temporary starvation of others	Initialize to global minimum pass
Waking process with stale pass	Pass value not adjusted on wake-up	Burst of service; unfairness	Clamp pass to global minimum on wake-up
Blocked process left in set	Implementation bug	Wasted scheduling decisions; incorrect pass updates	Remove blocked processes from the runnable set

CHAPTER 13

Deadlock and Its Relationship to Stride Scheduling

Definition of Deadlock

Deadlock is a state in which a set of processes are permanently blocked because each is waiting for a resource held by another process in the set. Deadlock is a resource-allocation phenomenon, not a CPU-scheduling phenomenon.

The Four Coffman Conditions

Deadlock requires four conditions to hold simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait[reference:26]. These conditions describe the resource-allocation graph, not the CPU scheduler.

Deadlock versus Starvation

Deadlock and starvation are different. In deadlock, processes are blocked and cannot proceed even if the CPU is available. In starvation, processes are runnable but are not selected by the scheduler. Stride Scheduling addresses CPU allocation among runnable processes; it does not address resource deadlock.

Does Stride Scheduling Prevent or Cause Deadlock?

Stride Scheduling neither prevents nor causes deadlock. Deadlock arises from resource-allocation policies, such as lock acquisition order and resource holding. The CPU scheduler determines which runnable process executes, but it does not control whether a process acquires or releases a lock. A process that is deadlocked is blocked, and Stride Scheduling will simply remove it from the runnable set or leave it blocked; the algorithm does not resolve the deadlock.

Why CPU Scheduling Alone Does Not Eliminate Deadlock

Deadlock is a property of the resource-allocation graph. Changing the CPU scheduling policy does not change the graph. To prevent or resolve deadlock, the operating system must use resource-allocation strategies such as lock ordering, timeout-based retry, deadlock detection, or recovery mechanisms.

Blocked Processes and the Runnable Set

In a correct implementation, a process blocked on a resource is removed from the runnable set. If it is not removed, the scheduler may waste decisions on a process that cannot run. This is an implementation issue, not a fundamental property of Stride Scheduling.

Deadlock Prevention, Avoidance, Detection, and Recovery

Operating systems use several strategies to handle deadlock: prevention (ensuring at least one Coffman condition cannot hold), avoidance (using algorithms such as the banker's algorithm), detection (maintaining a wait-for graph and searching for cycles), and recovery (terminating processes or preempting resources). These strategies are independent of the CPU scheduling algorithm.

Table 8. Deadlock Concepts and Operating-System Responses

Concept	Description	OS Response
Mutual exclusion	Resource cannot be shared simultaneously	Use sharable resources or spooling where possible
Hold and wait	Process holds one resource while waiting for another	Require all resources to be requested at once
No preemption	Resources cannot be forcibly reclaimed	Allow preemption of resources where feasible
Circular wait	Cyclic dependency among waiting processes	Impose a total ordering on resource acquisition
Deadlock detection	Identify cycles in the wait-for graph	Run periodic detection and recover

CHAPTER 14

Performance Analysis and Complexity

Scheduling Decision Complexity

The complexity of a scheduling decision depends on the data structure used to maintain the runnable set. A linear scan requires O(n) comparisons per decision, where n is the number of runnable processes. A min-heap provides O(1) access to the minimum and O(log n) insertion and deletion. A balanced search tree provides O(log n) search, insertion, and deletion.

Linear-Search Implementation

A linear search is simple and requires no additional data structure beyond an array or linked list. However, its O(n) per-decision cost becomes prohibitive for large n. It may be acceptable for small systems or for educational purposes.

Priority-Queue Implementation

A priority queue, typically implemented as a min-heap, is the natural data structure for Stride Scheduling. The process with the minimum pass value is at the root of the heap. When its pass value is updated, a decrease-key or increase-key operation restores the heap property in O(log n) time. This makes the priority queue implementation suitable for systems with many runnable processes.

Process Insertion and Removal

Insertion and removal are O(log n) in a heap or balanced tree, and O(1) (insertion) or O(n) (removal) in an unsorted array. The choice of data structure also affects memory requirements: a heap stores the processes in an array, while a tree requires pointers or an ordered array.

Context-Switching Overhead

Context-switching overhead is independent of the scheduling policy, but the frequency of context switches depends on the quantum length. A shorter quantum provides finer-grained fairness but increases overhead. The optimal quantum length is a trade-off between responsiveness and efficiency.

Memory Requirements

Each process requires storage for its ticket count, stride, and pass value. The runnable set requires storage proportional to the number of runnable processes. The memory overhead is modest and comparable to other scheduling algorithms.

Fairness Error over Finite Intervals

The difference between the actual service received and the theoretical proportional share is bounded by the maximum stride value and the quantum length. For a process with stride s, the pass value can deviate from the global

minimum by at most s. This bound provides a useful characterization of short-term fairness.

Impact of Workload Characteristics

Workload characteristics such as the number of processes, the distribution of ticket counts, and the frequency of blocking and wake-up events affect the algorithm's performance. Workloads with many short-lived processes require efficient insertion and removal. Workloads with highly skewed ticket distributions may produce long waiting times for low-ticket processes.

Table 9. Implementation Complexity Comparison

Operation	Array (Linear)	Min-Heap	Balanced BST
Find minimum	O(n)	O(1)	O(log n)
Insert	O(1)	O(log n)	O(log n)
Remove	O(n)	O(log n)	O(log n)
Update key	O(1) + O(n) rescan	O(log n)	O(log n)
Memory overhead	Low	Low–moderate	Moderate–high

CHAPTER 15

Charts and Graphical Analysis

This chapter presents five charts based on the illustrative example from Chapter 6. All data are explicitly defined and reproducible. Theoretical values are distinguished from simulated observations.



Figure 2. Theoretical relationship between ticket count and CPU share. The shares are proportional to ticket counts (total = 175 tickets).



Figure 3. Pass-value progression for P1, P2, and P3. The slopes are inversely proportional to the strides: P1 has the smallest stride and therefore the gentlest slope, while P3 has the largest stride and the steepest slope.



Figure 4. CPU scheduling timeline for the first 12 quanta in the illustrative example. The sequence is P1, P2, P3, P1, P1, P2, P1, P1, P2, P3, P1, P1.



Figure 5. Stride values for processes with different ticket counts. The stride is inversely proportional to the ticket count.



Figure 6. Comparison of theoretical and observed CPU shares after 12 scheduling decisions. The observed shares are close to the theoretical shares, with minor deviations expected in a short interval.

CHAPTER 16

Comparison with Other Scheduling Algorithms

This chapter compares Stride Scheduling with Round Robin, FCFS, Priority Scheduling, Completely Fair Scheduling (CFS), Rate Monotonic (RM), and Earliest Deadline First (EDF). The comparison is structured to highlight the different objectives and assumptions of general-purpose, proportional-share, and real-time scheduling algorithms.

Table 10. Comparison of Stride Scheduling with Other CPU Scheduling Algorithms

Criterion	Stride Scheduling	Round Robin	FCFS	Priority Scheduling	CFS (Linux)	Rate Monotonic	EDF
Scheduling principle	Select minimum pass; pass += K / tickets	Circular order with fixed quantum	First arrival runs first	Highest priority runs first	Select minimum vruntime	Shortest period = highest fixed priority	Earliest absolute deadline first
Objective	Proportional CPU share	Fairness, responsiveness	Simplicity, no starvation	Priority differentiation	Fair CPU share	Meet deadlines (periodic tasks)	Meet deadlines (dynamic)
Preemption	Yes, at quantum boundaries	Yes	No	Yes (typically)	Yes	Yes	Yes
Resource-sharing model	Proportional share via tickets	Equal share among runnable	Arrival order	Priority order	Weighted fair share via nice values	Fixed-priority real-time	Dynamic-priority real-time
Deadline support	None	None	None	None	None	Yes (with utilization bound)	Yes (optimal for uniprocessor)
Starvation risk	Low for positive tickets; possible with very low tickets	Low	None	High for low-priority tasks	Low	High for low-priority tasks	Low under feasible load
Computational overhead	O(log n) with heap/BST; O(n) linear	O(1)	O(1)	O(log n) or O(n)	O(log n) with RB-tree	O(n) or O(log n)	O(log n)
Typical use case	Weighted resource allocation, research	General-purpose time sharing	Batch systems	Systems with clear priority classes	Desktop and server Linux	Embedded real-time systems	Real-time systems with dynamic deadlines
Main advantage	Precise proportional sharing	Simple and fair	Simple, no starvation	Respects priorities	Responsive and fair	Predictable, simple analysis	Optimal utilization
Main limitation	No deadline support; multicore challenges	No priority differentiation	Convoy effect	Starvation of low priorities	Heuristic complexity (pre-EEVDF)	Low utilization bound	Complex under overload

The table makes clear that Stride Scheduling belongs to the proportional-share family and shares some conceptual similarities with CFS, which uses a virtual runtime mechanism loosely based on the stride concept[reference:27].

However, RM and EDF belong to the real-time family and have fundamentally different objectives. They should not be treated as interchangeable with Stride Scheduling.

CHAPTER 17

Applications and Practical Use Cases

Weighted Resource Allocation

Stride Scheduling is appropriate for systems that require weighted allocation of a resource. Examples include allocating CPU time among users, processes, or containers according to configured weights.

Operating-System Scheduling Research

The algorithm is widely used in scheduling research as a baseline proportional-share mechanism. Its determinism and mathematical tractability make it suitable for simulation studies and comparative evaluations.

Educational Simulations

Stride Scheduling is an excellent pedagogical tool. Its core formulas are simple enough for students to simulate by hand, yet its behavior illustrates important concepts such as virtual time, fairness, and the trade-off between short-term and long-term behavior.

CPU-Share Experiments

Researchers can use Stride Scheduling to conduct controlled experiments on CPU sharing. The ticket abstraction allows precise specification of resource entitlements, and the deterministic selection makes experimental results reproducible.

Fair Resource Distribution among Competing Tasks

In systems where multiple tasks compete for a resource and no task has a hard deadline, Stride Scheduling can provide a fair distribution according to specified shares. This is particularly useful when tasks have different importance or different resource requirements.

Virtualized and Multi-Tenant Environments

With appropriate adaptation, Stride Scheduling can be used to distribute CPU capacity among virtual machines or tenants according to contractual entitlements. However, modern hypervisors and container orchestrators typically use more complex mechanisms that account for multicore scheduling, I/O, and memory effects. The original algorithm should be regarded as a conceptual foundation rather than a drop-in solution for modern cloud infrastructure.

> **Clarification:** Stride Scheduling is not the default scheduling algorithm in modern general-purpose operating systems. Linux's Completely Fair Scheduler (CFS) and its successor EEVDF are the default CPU schedulers in contemporary Linux kernels. Stride Scheduling is best understood as an influential proportional-share algorithm with significant pedagogical and research value.

CHAPTER 18

Experimental Evaluation

Experimental Objectives

The objective of this evaluation is to demonstrate the proportional-share behavior of Stride Scheduling using a fully reproducible simulation. The evaluation is hardware-independent and does not claim to represent measurements from a physical system.

Simulation Assumptions

- Single processor with a fixed quantum.
- All processes are continuously runnable.
- Tickets are positive integers.
- Stride constant `K = 10000` .
- Initial pass values are zero.
- Tie-breaking: smaller stride wins; if equal, lower process ID wins.

Process Configuration and Tickets

Table 11. Experimental Configuration

Process	Tickets	Stride	Initial Pass
P1	100	100	0
P2	50	200	0
P3	25	400	0

Metrics and Expected Observations

The primary metric is the observed CPU share over a fixed number of scheduling decisions. The expected observation is that the observed shares converge toward the theoretical shares as the number of decisions increases. The secondary metric is the maximum absolute deviation from the theoretical share.

Results and Discussion

The results of the first 12 decisions were presented in Chapter 6. As the number of decisions increases, the deviation decreases. This behavior is consistent with the algorithm's theoretical properties. The simulation can be extended to thousands of decisions to observe convergence.

Reproducibility

The simulation can be reproduced using a short Python script that implements the pseudocode from Chapter 7. The script should maintain a list of processes, each with a ticket count, stride, and pass value; at each step, it selects the process with the minimum pass value, increments its pass value by its stride, and records the selection. The data in this chapter can be verified by running such a script.

CHAPTER 19

Possible Improvements and Future Research

Numerical Stability

Periodic normalization of pass values can prevent numerical drift. Normalization involves subtracting the global minimum pass value from all pass values, or rescaling pass values when they grow too large. The original work discusses quantization errors and suggests infrequent normalization[reference:28].

Pass-Value Normalization

Normalization can be performed when the minimum pass value exceeds a threshold. This keeps pass values within a bounded range and reduces the risk of overflow. The normalization operation is O(n) but can be amortized over many scheduling decisions.

Improved Initialization

Initializing new processes to the current global minimum pass value is a simple and effective policy. More sophisticated policies could consider the process's expected workload or its creator's pass value. These policies remain an area of open research.

Efficient Runnable-Process Data Structures

Using a min-heap or balanced search tree provides O(log n) scheduling decisions. Further improvements could include using bucketed priority queues or radix heaps for integer pass values, which can provide O(1) amortized operations under certain conditions.

Multicore Fairness

Maintaining global proportional fairness across multiple cores remains a challenging research problem. Techniques such as global virtual time, per-CPU pass-value synchronization, and hierarchical stride scheduling are promising directions[reference:29].

Dynamic Ticket Allocation

Allowing tickets to be adjusted dynamically introduces new challenges: when a process's ticket count changes, its stride must be recomputed, and its pass value may need to be adjusted to avoid unfairness. The original work discusses ticket transfers and currencies as mechanisms for flexible allocation[reference:30].

Workload-Aware Resource Sharing

Future schedulers could use workload characteristics to adjust ticket allocations automatically. For example, a scheduler could detect that a process is interactive and temporarily increase its tickets to reduce latency, while still preserving long-term proportional fairness.

Latency and Responsiveness

Combining proportional-share scheduling with latency-awareness is an active area of research. Linux's CFS includes heuristics for sleeper fairness and wake-up granularity, although these heuristics have been criticized for their complexity[reference:31]. Future work could design cleaner mechanisms for achieving both fairness and responsiveness.

Hybrid Scheduling Approaches

Hybrid approaches that combine proportional-share scheduling with real-time scheduling may be appropriate for systems that have both throughput-oriented and deadline-sensitive tasks. Hierarchical scheduling frameworks provide a natural way to compose such policies.

CHAPTER 20

Conclusion

Stride Scheduling is a deterministic proportional-share CPU scheduling algorithm that provides a mathematically elegant mechanism for allocating processor time in proportion to ticket counts. Its operating principle is simple: each process has a stride inversely proportional to its ticket count, and the scheduler always selects the runnable process with the minimum pass value, advancing that pass value by the process's stride after each quantum. This mechanism ensures that, over sufficiently long intervals, the CPU share received by each process converges to its ticket proportion, provided that tickets are positive and the runnable set is stable.

The algorithm's principal strengths are its precise proportional sharing, its deterministic behavior, its relatively simple mathematical foundations, and its flexibility through the ticket abstraction. These strengths make it valuable for weighted resource allocation, operating-system research, educational simulations, and controlled CPU-share experiments. The algorithm also provides conceptual foundations for later proportional-share schedulers, most notably Linux's Completely Fair Scheduler, which uses a virtual runtime mechanism loosely inspired by the stride concept.

However, Stride Scheduling has important limitations. Its fairness guarantees depend on correct ticket allocation, appropriate initialization policies, and careful handling of process arrival, departure, blocking, and wake-up. Numerical precision can drift over long periods, and very low ticket allocations can produce long waiting times that may be perceived as starvation. The algorithm does not provide deadline guarantees and is not a substitute for real-time scheduling algorithms such as Rate Monotonic or Earliest Deadline First. Extending Stride Scheduling to multicore systems requires additional design decisions concerning run-queue organization, synchronization, migration, and global versus per-CPU pass-value management. A basic single-processor implementation does not automatically provide correct global proportional fairness on multiple processors.

In conclusion, Stride Scheduling is a well-understood and influential proportional-share algorithm whose advantages are strongest in environments where weighted CPU sharing is the primary objective and where the workload is relatively stable. It is not a universal solution to all scheduling problems, and its limitations must be understood and addressed in any practical deployment. Its pedagogical value is considerable, and its conceptual legacy continues to shape modern scheduler design.

CHAPTER 21

References

1. C. A. Waldspurger and W. E. Weihl, "Lottery scheduling: Flexible proportional-share resource management," in *Proceedings of the First Symposium on Operating Systems Design and Implementation (OSDI)*, Monterey, CA, USA, Nov. 1994, pp. 1–11[reference:32].
2. C. A. Waldspurger, "Lottery and stride scheduling: Flexible proportional-share resource management," Ph.D. dissertation, Dept. Elect. Eng. Comput. Sci., Massachusetts Institute of Technology, Cambridge, MA, USA, Sep. 1995[reference:33].
3. C. A. Waldspurger and W. E. Weihl, "Stride scheduling: Deterministic proportional-share resource management," MIT Laboratory for Computer Science, Cambridge, MA, USA, Technical Memorandum MIT/LCS/TM-528, Jun. 1995[reference:34].
4. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
5. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Madison, WI, USA: Arpaci-Dusseau Books, 2018.
6. C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
7. G. C. Buttazzo, "Rate monotonic vs. EDF: Judgment day," *Real-Time Systems*, vol. 29, no. 1, pp. 5–26, Jan. 2005[reference:35].
8. E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, Jun. 1971.
9. D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.
10. H. Li, "CPU scheduling: CFS to EEVDF," Lecture notes, CS 5264/4224 & ECE 5414/4414, Virginia Tech, Feb. 2026[reference:36].
11. W. J. Lloyd, "The Completely Fair Scheduler (CFS)," TCSS 422: Operating Systems, University of Washington – Tacoma, Apr. 2023[reference:37].
12. C. A. Waldspurger and W. E. Weihl, "Stride scheduling: Deterministic proportional-share resource management," MIT Laboratory for Computer Science, Technical Memorandum MIT/LCS/TM-528, Jun. 1995. (Cited for the discussion of dynamic client-set changes and fixed-point precision.)[reference:38]
13. Project 2b: xv6 Scheduler, University of Wisconsin–Madison. (Cited for the discussion of pass-value initialization and starvation-like behavior in educational implementations.)[reference:39]
14. Proportional Share Scheduling employing Performance-aware Virtual Time in Multiprocessor Systems, *Semantic Scholar*. (Cited for the discussion of multiprocessor proportional-share fairness.)[reference:40]

— End of Paper —